**Institut Supérieur de l'Informatique**

*Niveau 2IDISC*

*Année universitaire : 2025/2026*

# TP 4

# Sécurisation d'une Architecture IoT basée sur MQTT

## Compte Rendu Complet

| Environnement | Debian Linux + ESP32 hardware réel |
|---|---|
| Broker MQTT | Mosquitto 2.0.21 |
| Réseau | WiFi LAN 172.20.10.x (hotspot) |
| IP Debian (Broker) | 172.20.10.2 |
| IP ESP32 (Capteur) | 172.20.10.3 |
| Outils | PlatformIO, Wireshark, tcpdump, OpenSSL |
| Date | 01 Avril 2026 |

*Aziz Harzallah*

# Table des Matières

1. Introduction et Contexte
2. Partie 1 — Déploiement de l'Architecture IoT
   2.1 Installation de Mosquitto
   2.2 Configuration du broker (sans sécurité)
   2.3 Programmation ESP32 avec PlatformIO
   2.4 Vérification via subscriber terminal
3. Partie 2 — Analyse des Vulnérabilités
   3.1 Capture du trafic avec tcpdump et Wireshark
   3.2 Analyse des paquets MQTT
   3.3 Réponses aux questions et identification des risques
4. Partie 3 — Mise en Place de l'Authentification
   4.1 Création du fichier de mots de passe
   4.2 Configuration du broker avec authentification
   4.3 Mise à jour du client ESP32
   4.4 Tests de validation
5. Partie 4 — Implémentation TLS
   5.1 Génération des certificats SSL/TLS
   5.2 Configuration du broker pour TLS
   5.3 Mise à jour de l'ESP32 pour TLS
   5.4 Vérification avec Wireshark
   5.5 Benchmark latence et débit
6. Partie 5 — Analyse Comparative et Recommandations
7. Questions Bonus
8. Conclusion

# 1. Introduction et Contexte

Les systèmes IoT sont massivement déployés dans les environnements industriels, médicaux et urbains. Ces systèmes utilisent souvent le protocole MQTT (Message Queuing Telemetry Transport), un protocole léger basé sur le modèle publish/subscribe. De nombreuses implémentations manquent de mécanismes de sécurité adéquats, exposant les données à des menaces variées.

Ce TP met en oeuvre une démarche complète : déploiement d'une architecture vulnérable, analyse des failles, puis sécurisation progressive par authentification et chiffrement TLS.

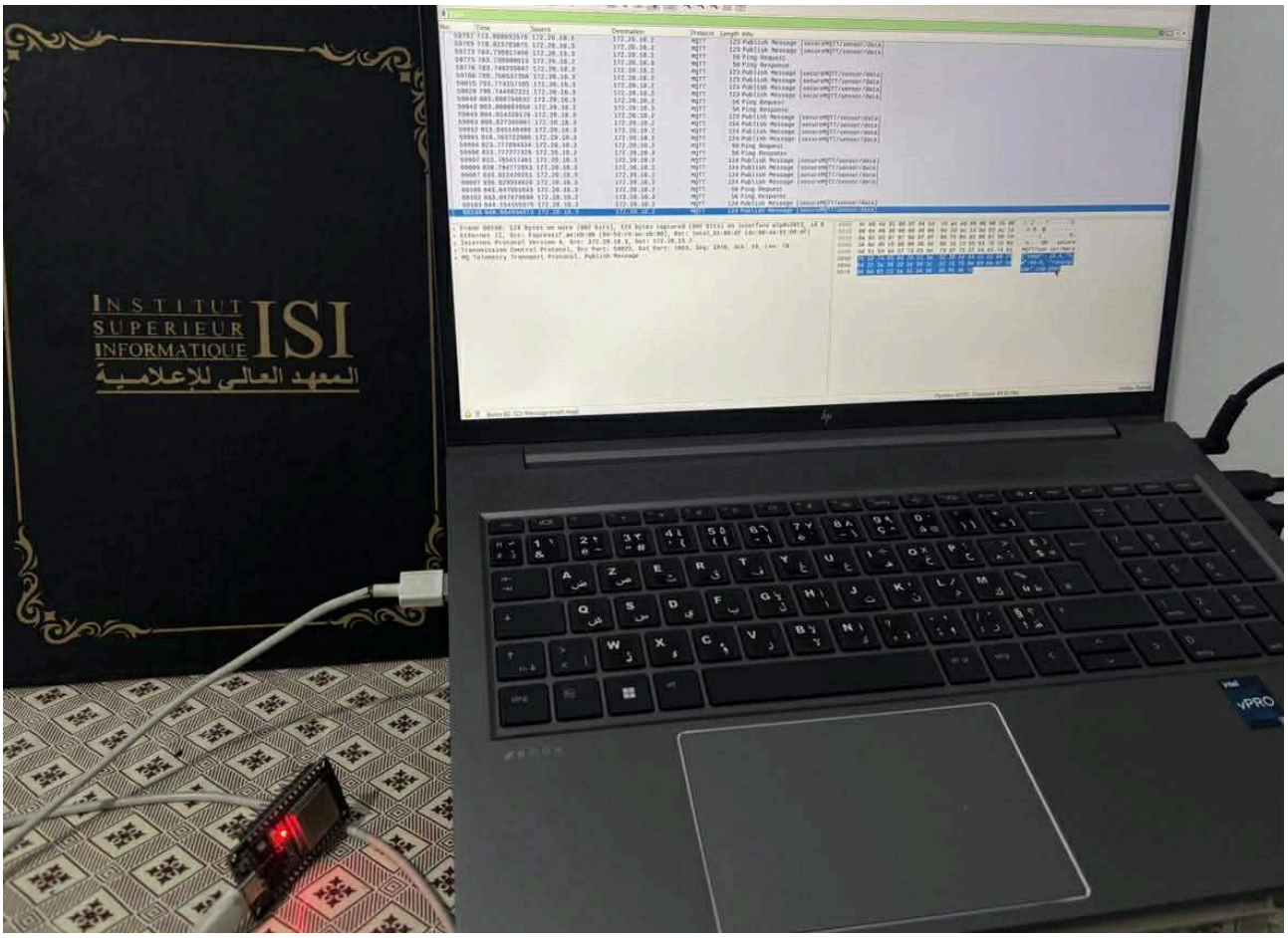
## 1.1 Objectifs pédagogiques

- Déployer un environnement IoT fonctionnel utilisant MQTT
- Identifier les vulnérabilités d'une architecture IoT non sécurisée
- Implémenter authentification, TLS et mTLS
- Comparer les performances avant et après sécurisation
- Proposer des améliorations avancées (Zero Trust, IDS, segmentation réseau)

## 1.2 Architecture générale du TP

| Composant | Description |
|---|---|
| ESP32 (172.20.10.3) | Capteur IoT réel — publie données temp/humidité toutes les 5 secondes sur iot/sensor/data |
| Broker Mosquitto (172.20.10.2) | Serveur MQTT sur Debian — reçoit et redistribue les messages sur le port 1883 (puis 8883 avec TLS) |
| Subscriber CLI | mosquitto_sub sur PC Debian — reçoit et affiche les messages publiés par l'ESP32 |
| Réseau WiFi LAN | Hotspot 172.20.10.x — ESP32 et Debian connectés au même réseau |
| Wireshark / tcpdump | Capture et analyse du trafic réseau sur l'interface WiFi wlp0s20f3 |

### 1.3 Environnement de travail




## 2. Partie 1 — Déploiement de l'Architecture IoT

### 2.1 Installation de Mosquitto et des outils

Installation de tous les paquets nécessaires sur la machine Debian :

```
sudo apt update && sudo apt upgrade -y
sudo apt install mosquitto mosquitto-clients -y
sudo apt install wireshark tcpdump openssl -y
sudo usermod -aG wireshark $USER
```

Activation et démarrage du service Mosquitto :

```
sudo systemctl enable mosquitto
sudo systemctl restart mosquitto
sudo systemctl status mosquitto
```

```
primaryos@ninja:~$ sudo systemctl restart mosquitto
primaryos@ninja:~$ sudo systemctl status mosquitto
● mosquitto.service - Mosquitto MQTT Broker
     Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-31 23:31:42 CET; 1min 27s ago
 Invocation: 284ede14bc3643d29a1561c7e4ae6c0e
       Docs: man:mosquitto.conf(5)
             man:mosquitto(8)
    Process: 26064 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=>
    Process: 26065 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exite>
    Process: 26066 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SU>
    Process: 26069 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, s>
   Main PID: 26072 (mosquitto)
      Tasks: 1 (limit: 37672)
     Memory: 1.1M (peak: 2.3M)
        CPU: 74ms
```

## 2.2 Configuration du Broker (Phase 1 — sans sécurité)

Le fichier de configuration initial permet à tous les clients de se connecter sans authentification :

```
sudo nano /etc/mosquitto/conf.d/tp4.conf
```

Contenu du fichier /etc/mosquitto/conf.d/tp4.conf :

```
listener 1883
allow_anonymous true
```

Contenu du fichier principal /etc/mosquitto/mosquitto.conf :

```
persistence true
persistence_location /var/lib/mosquitto/
log_dest file /var/log/mosquitto/mosquitto.log
include_dir /etc/mosquitto/conf.d
```

---

Vérification que le port 1883 est actif et configuration du pare-feu :

```
ss -tlnp | grep 1883
sudo ufw allow 1883 && sudo ufw allow 8883 && sudo ufw enable
```

```
primaryos@ninja:~$ ss -tlnp | grep 1883
LISTEN 0      100          0.0.0.0:1883       0.0.0.0:*
LISTEN 0      100             [::]:1883          [::]:*
```

## 2.3 Programmation de l'ESP32 avec PlatformIO

Configuration du projet PlatformIO — fichier platformio.ini :

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps = knolleary/PubSubClient@^2.8
```

[

Code source ESP32 — Partie 1 sans sécurité (src/main.cpp) :

```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid        = "NOM_WIFI";
const char* password    = "MOT_DE_PASSE";
const char* mqtt_server = "172.20.10.2";  // IP Debian
const int   mqtt_port   = 1883;
WiFiClient espClient;
PubSubClient client(espClient);
```

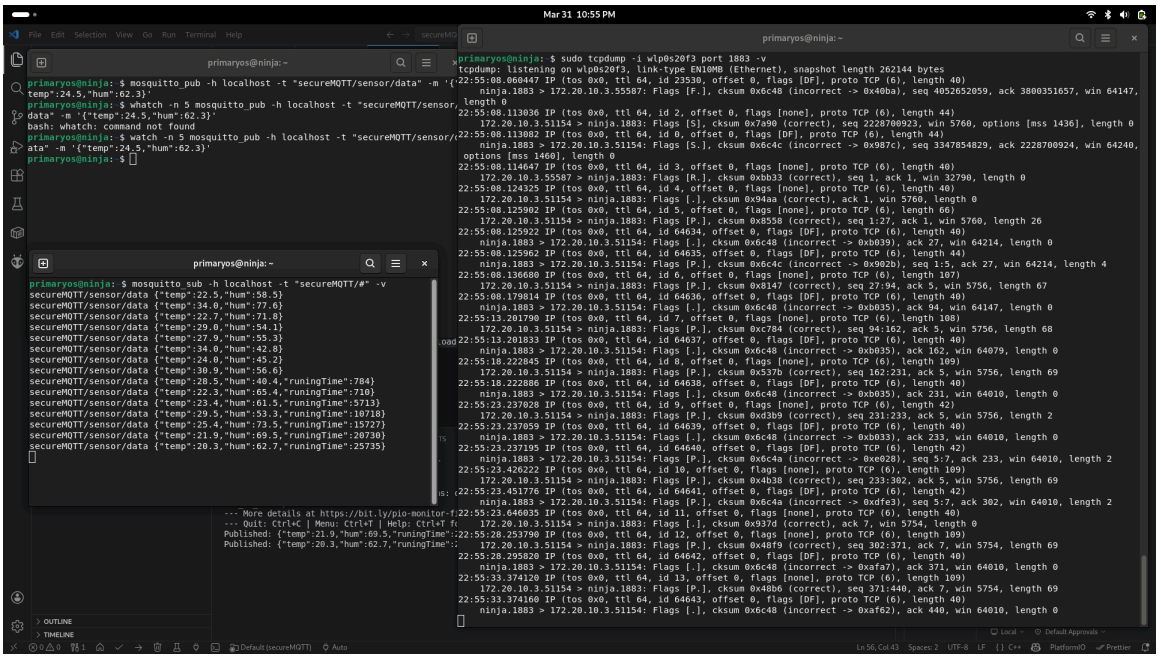
```
void connectMQTT() {
  while (!client.connected()) {
    if (client.connect("esp32_sensor")) {
      Serial.println("Connected!");
    } else { delay(2000); }
  }
}
```

```
void loop() {
  if (!client.connected()) connectMQTT();
  client.loop();
  float temp = 20.0 + random(0,150)/10.0;
  float hum  = 40.0 + random(0,400)/10.0;
  String payload = "{\"temp\":" + String(temp,1)
                 + ",\"hum\":" + String(hum,1) + "}";
  client.publish("iot/sensor/data", payload.c_str());
  Serial.println("Published: " + payload);
  delay(5000);
}
```

## 2.4 Vérification via Subscriber Terminal

Réception des messages vérifiée depuis le terminal Debian :

```
mosquitto_sub -h localhost -t "iot/#" -v
```




# 3. Partie 2 — Analyse des Vulnérabilités

## 3.1 Capture du trafic avec tcpdump et Wireshark

Le trafic MQTT entre l'ESP32 (172.20.10.3) et le broker Debian (172.20.10.2) a été capturé sur l'interface WiFi wlp0s20f3.

### Sauvegarde de la capture pour analyse Wireshark :

```
sudo tcpdump -i wlp0s20f3 port 1883 -w ~/part2_vulnerable.pcap
# Laisser tourner 30 secondes puis Ctrl+C
wireshark ~/part2_vulnerable.pcap
```

Filtre appliqué dans Wireshark pour isoler le trafic MQTT :

```
tcp.port == 1883
```

## 3.2 Analyse des paquets MQTT capturés

### Paquet CONNECT — ESP32 vers broker :

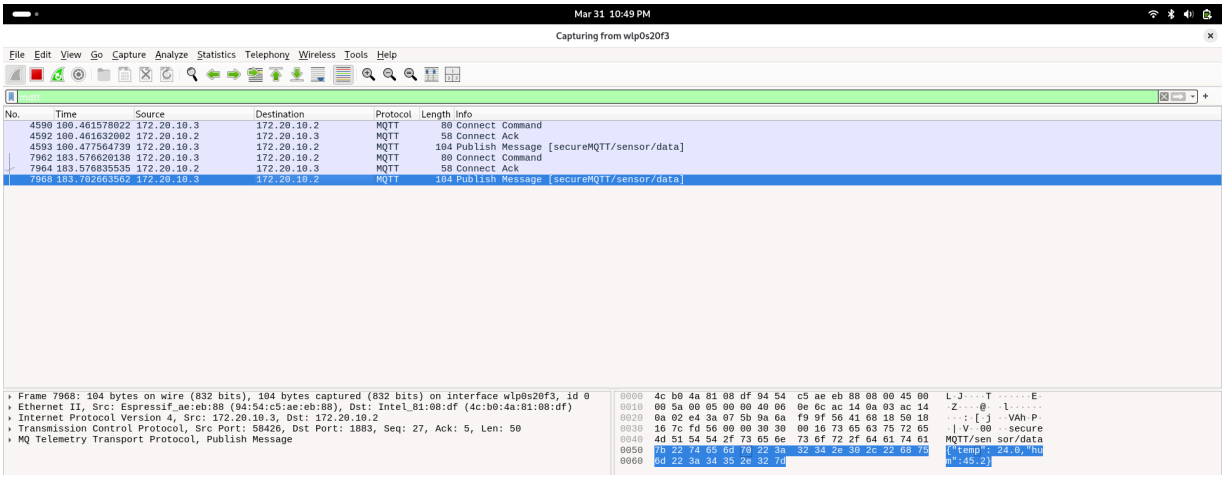
En développant la couche MQ Telemetry Transport Protocol dans Wireshark :

```
MQ Telemetry Transport Protocol, Connect Command
  Protocol Name: MQTT
  Version: 4 (MQTT 3.1.1)
  ClientId: esp32_sensor      <- visible en clair !
  Username: (vide)            <- aucune authentification !
  Password: (vide)            <- aucun mot de passe !
```

### Paquet PUBLISH — données capteur :

En développant la couche MQ Telemetry Transport Protocol d'un paquet PUBLISH :

```
MQ Telemetry Transport Protocol, Publish Message
  Topic Length: 16
  Topic: iot/sensor/data                <- visible en clair !
  Message: {"temp":24.5,"hum":62.3}     <- visible en clair !
```




## 3.3 Réponses aux questions et identification des risques

| Question | Réponse et observation |
|---|---|
| Les données sont-elles en clair ? | OUI — topic et payload lisibles dans tcpdump et Wireshark sans aucun outil de déchiffrement |
| Peut-on identifier les topics publiés ? | OUI —secureMQTT/sensor/data visible dans chaque paquet PUBLISH capturé |
| Pub/sub sans authentification ? | OUI — allow_anonymous true, n'importe quel client peut se connecter et publier librement |
| Quels types d'attaques sont possibles ? | Sniffing passif, injection de messages, MitM, usurpation d'identité, DoS par flood |

Risques identifiés dans la configuration vulnérable :

- Sniffing passif : tout observateur sur le réseau WiFi peut lire toutes les données en temps réel
- Injection de messages : n'importe qui peut publier de fausses données sur secureMQTT/sensor/data
- Usurpation d'identité : n'importe quel clientId est accepté sans vérification
- Man-in-the-Middle : le trafic peut être intercepté et modifié sans détection
- Attaque DoS : flood de connexions anonymes pouvant saturer le broker Mosquitto
- Accès non autorisé : subscription sur secureMQTT/# possible sans restriction

# 4. Partie 3 — Mise en Place de l'Authentification

## 4.1 Création du fichier de mots de passe

Un fichier de mots de passe hashés (algorithme PBKDF2-SHA512) a été créé pour le broker :

```
sudo mkdir -p /etc/mosquitto/auth
```

```
sudo mosquitto_passwd -c /etc/mosquitto/auth/passwd esp32
# Saisir le mot de passe : esp32pass
sudo mosquitto_passwd /etc/mosquitto/auth/passwd admin
# Saisir le mot de passe : adminpass
```

Vérification du fichier passwd (mots de passe hashés) :

```
sudo cat /etc/mosquitto/auth/passwd
```

```
primaryos@ninja:~$ sudo mkdir -p /etc/mosquitto/auth
[sudo] password for primaryos:
primaryos@ninja:~$ sudo mosquitto_passwd -c /etc/mosquitto/auth/passwd esp32
Password:
Reenter password:
primaryos@ninja:~$ sudo mosquitto_passwd /etc/mosquitto/auth/passwd admin
Password:
Reenter password:
Error: Passwords do not match.
primaryos@ninja:~$ sudo mosquitto_passwd /etc/mosquitto/auth/passwd admin
Password:
Reenter password:
primaryos@ninja:~$ sudo cat /etc/mosquitto/auth/passwd
esp32:$7$101$1VzFZZmhfVezUuDC$7MArSpf97PaC3cPFxhrOGTdtus7Klq6m0pecLQAh9YWOe/NkWQ
59GK97bJnGWO48ZY8lTvOIaOEa6Gfjt+IH7A==
admin:$7$101$mYyHp0fQEWZALoVi$LpdNiHMoNL8kslOtaqBDImdN1ce/hiQqPOiQGavG2Ha2qvJ4OQ
OM92AYlVIopzSPRIbkn+JsQ1+OHuT9VEW4zQ==
primaryos@ninja:~$
```

Correction des permissions pour que le processus Mosquitto puisse lire le fichier :

```
sudo chown mosquitto:mosquitto /etc/mosquitto/auth/passwd
sudo chmod 640 /etc/mosquitto/auth/passwd
sudo chown mosquitto:mosquitto /etc/mosquitto/auth
sudo chmod 750 /etc/mosquitto/auth
```

```
primaryos@ninja: ~
rimaryos@ninja:~$ sudo systemctl restart mosquitto
sudo] password for primaryos:
ob for mosquitto.service failed because the control process exited with error code.
ee "systemctl status mosquitto.service" and "journalctl -xeu mosquitto.service" for details.
rimaryos@ninja:~$ sudo journalctl -xeu mosquitto.service | tail -20
  Defined-By: systemd
  Support: https://www.debian.org/support

  Automatic restarting of the unit mosquitto.service has been scheduled, as the result for
  the configured Restart= setting for the unit.
ar 31 23:30:05 ninja systemd[1]: mosquitto.service: Start request repeated too quickly.
ar 31 23:30:05 ninja systemd[1]: mosquitto.service: Failed with result 'exit-code'.
  Subject: Unit failed
  Defined-By: systemd
  Support: https://www.debian.org/support

  The unit mosquitto.service has entered the 'failed' state with result 'exit-code'.
ar 31 23:30:05 ninja systemd[1]: Failed to start mosquitto.service - Mosquitto MQTT Broker.
  Subject: A start job for unit mosquitto.service has failed
  Defined-By: systemd
  Support: https://www.debian.org/support

  A start job for unit mosquitto.service has finished with a failure.

  The job identifier is 12528 and the job result is failed.
rimaryos@ninja:~$ # Give mosquitto user access to the passwd file
udo chown mosquitto:mosquitto /etc/mosquitto/auth/passwd
udo chmod 640 /etc/mosquitto/auth/passwd

 Also fix the directory
udo chown mosquitto:mosquitto /etc/mosquitto/auth
udo chmod 750 /etc/mosquitto/auth
rimaryos@ninja:~$ sudo systemctl restart mosquitto
rimaryos@ninja:~$
```

## 4.2 Configuration du Broker avec Authentification

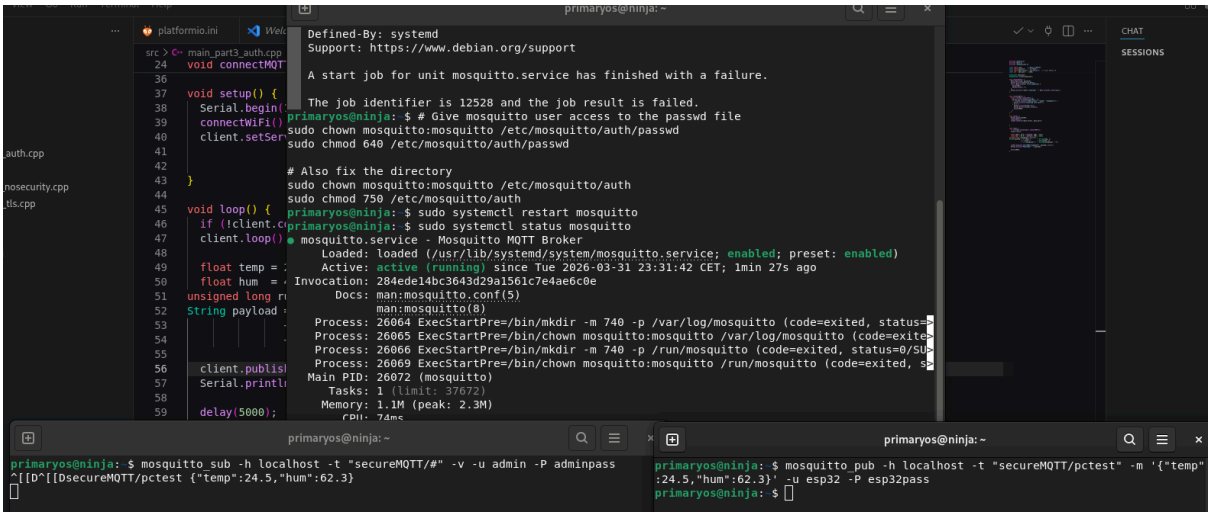
Le fichier tp4.conf a été mis à jour pour désactiver les connexions anonymes :

```
sudo nano /etc/mosquitto/conf.d/tp4.conf
```

Contenu du fichier tp4.conf (Phase 3 — avec authentification) :

```
listener 1883
allow_anonymous false
password_file /etc/mosquitto/auth/passwd
```

```
sudo systemctl restart mosquitto
sudo systemctl status mosquitto
```




## 4.3 Mise à Jour du Code ESP32

La fonction connectMQTT() a été modifiée pour inclure les identifiants :

```
void connectMQTT() {
  while (!client.connected()) {
    Serial.print("Connecting MQTT...");
    if (client.connect("esp32_device", "esp32", "esp32pass")) {
      Serial.println("Connected with auth!");
    } else {
      Serial.print("Failed rc=");
      Serial.println(client.state());
      delay(2000);
    }
  }
}
```

Les trois paramètres de client.connect() signifient :

- esp32_device : identifiant unique du device (clientId — libre)
- esp32 : username défini dans le fichier /etc/mosquitto/auth/passwd
- esp32pass : mot de passe correspondant à ce username

## 4.4 Tests de Validation

### Test 1 — Connexion refusée sans identifiants :

```
mosquitto_sub -h localhost -t "secureMQTT/#" -v
```

```
primaryos@ninja:~$ mosquitto_sub -h localhost -t "secureMQTT/#" -v
Connection error: Connection Refused: not authorised.
primaryos@ninja:~$ mosquitto_sub -h localhost -t "secureMQTT/#" -v
Connection error: Connection Refused: not authorised.
```

### Test 2 — Connexion refusée avec mauvais mot de passe :

```
mosquitto_sub -h localhost -t "secureMQTT/#" -v -u admin -P wrongpass=adminPass
```

```
primaryos@ninja:~$ mosquitto_sub -h localhost -t "secureMQTT/#" -v -u admin -P adminPass
Connection error: Connection Refused: not authorised.
```

### Test 3 — Connexion acceptée avec identifiants corrects :

```
mosquitto_sub -h localhost -t "iot/#" -v -u admin -P adminpass
```

```
primaryos@ninja:~$ mosquitto_sub -h localhost -t "secureMQTT/#" -v -u admin -P adminpass
secureMQTT/esp32Auth {"temp":30.1,"hum":76.5,"runingTime":712}
secureMQTT/esp32Auth {"temp":32.0,"hum":59.2,"runingTime":5716}
secureMQTT/esp32Auth {"temp":28.3,"hum":58.8,"runingTime":1264}
secureMQTT/esp32Auth {"temp":30.7,"hum":45.3,"runingTime":6267}
secureMQTT/esp32Auth {"temp":34.5,"hum":42.6,"runingTime":11271}
secureMQTT/esp32Auth {"temp":27.1,"hum":59.9,"runingTime":16280}
secureMQTT/esp32Auth {"temp":29.3,"hum":62.7,"runingTime":21282}
```

### Test 4 — Connexion du esp32 refusée sans la mise à jour de la fonction connectMQTT:

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4
Connecting WiFi.
WiFi connected: 172.20.10.3
Connecting MQTT...Failed rc=5
Connecting MQTT...Failed rc=5
Connecting MQTT...Failed rc=5
Connecting MQTT...Failed rc=5
```

| Test effectué | Commande | Résultat |
|---|---|---|
| Sans identifiants | mosquitto_sub (sans -u -P) | REFUSE |
| Mauvais mot de passe | -u admin -P wrongpass | REFUSE |
| Identifiants corrects | -u admin -P adminpass | ACCEPTE |
| ESP32 avec auth | client.connect("esp32_device", "esp32","esp32pass") | ACCEPTE |

# 5. Partie 4 — Implémentation TLS

## 5.1 Génération des Certificats SSL/TLS

Les certificats ont été générés avec OpenSSL dans /etc/mosquitto/certs/ :

```
sudo mkdir -p /etc/mosquitto/certs
cd /etc/mosquitto/certs
```

### Etape 1 — Certificat de l'Autorité de Certification (CA) :

```
sudo openssl genrsa -out ca.key 2048
sudo openssl req -new -x509 -days 1826 \
  -key ca.key -out ca.crt \
  -subj "/CN=IoT-CA/O=TP4/C=TN"
```

### Etape 2 — Certificat serveur (broker Mosquitto) :

```
sudo openssl genrsa -out server.key 2048
sudo openssl req -new -key server.key -out server.csr \
  -subj "/CN=localhost/O=TP4/C=TN"
sudo openssl x509 -req -days 360 \
  -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt
```

```
primaryos@ninja:~$ sudo mkdir -p /etc/mosquitto/certs
cd /etc/mosquitto/certs
[sudo] password for primaryos:
primaryos@ninja:/etc/mosquitto/certs$ # Generate CA private key
sudo openssl genrsa -out ca.key 2048

# Generate CA certificate
sudo openssl req -new -x509 -days 1826 \
  -key ca.key \
  -out ca.crt \
  -subj "/CN=IoT-CA/O=TP4/C=TN"
primaryos@ninja:/etc/mosquitto/certs$ # Server private key
sudo openssl genrsa -out server.key 2048

# Server certificate signing request
sudo openssl req -new \
  -key server.key \
  -out server.csr \
  -subj "/CN=172.20.10.2/O=TP4/C=TN"
# ↑ use your actual Debian IP

# Sign server certificate with CA
sudo openssl x509 -req -days 360 \
  -in server.csr \
  -CA ca.crt \
  -CAkey ca.key \
  -CAcreateserial \
  -out server.crt

# Fix permissions
sudo chmod 644 /etc/mosquitto/certs/*
sudo chown mosquitto:mosquitto /etc/mosquitto/certs/*
Certificate request self-signature ok
subject=CN=172.20.10.2, O=TP4, C=TN
primaryos@ninja:/etc/mosquitto/certs$ sudo nano /etc/mosquitto/conf.d/tp4.conf
primaryos@ninja:/etc/mosquitto/certs$ sudo nano /etc/mosquitto/conf.d/tp4.conf
primaryos@ninja:/etc/mosquitto/certs$ sudo systemctl restart mosquitto
```

### Etape 3 — Correction des permissions :

```
sudo chown mosquitto:mosquitto /etc/mosquitto/certs/*
sudo chmod 644 /etc/mosquitto/certs/ca.crt
sudo chmod 644 /etc/mosquitto/certs/server.crt
sudo chmod 600 /etc/mosquitto/certs/server.key
```

Vérification des certificats générés :

```
sudo openssl x509 -in server.crt -text -noout | grep -E "Subject:|Issuer:|Not"
sudo ls -la /etc/mosquitto/certs/
```

```
primaryos@ninja:~$ sudo ls -la /etc/mosquitto/certs/
[sudo] password for primaryos:
Sorry, try again.
[sudo] password for primaryos:
total 36
drwxr-xr-x 2 root      root      4096 Apr  1 00:02 .
drwxr-xr-x 6 root      root      4096 Mar 31 23:10 ..
-rw-r--r-- 1 mosquitto mosquitto 1180 Apr  1 00:00 ca.crt
-rw-r--r-- 1 mosquitto mosquitto 1704 Apr  1 00:00 ca.key
-rw-r--r-- 1 mosquitto mosquitto   41 Apr  1 00:02 ca.srl
-rw-r--r-- 1 mosquitto mosquitto  130 Mar 21  2025 README
-rw-r--r-- 1 mosquitto mosquitto 1164 Apr  1 00:02 server.crt
-rw-r--r-- 1 mosquitto mosquitto  932 Apr  1 00:02 server.csr
-rw------- 1 mosquitto mosquitto 1704 Apr  1 00:02 server.key
```

## 5.2 Configuration du Broker pour TLS

Le fichier tp4.conf a été mis à jour pour activer TLS sur le port 8883 tout en conservant le port 1883 :

```
# /etc/mosquitto/conf.d/tp4.conf
listener 1883
allow_anonymous false
password_file /etc/mosquitto/auth/passwd

listener 8883
cafile   /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile  /etc/mosquitto/certs/server.key
tls_version tlsv1.2
allow_anonymous false
password_file /etc/mosquitto/auth/passwd
```

```
sudo systemctl restart mosquitto
ss -tlnp | grep -E '1883|8883'
```

```
primaryos@ninja:~$ ss -tlnp | grep -E '1883|8883'
LISTEN 0      100          0.0.0.0:8883       0.0.0.0:*

LISTEN 0      100          0.0.0.0:1883       0.0.0.0:*

LISTEN 0      100             [::]:8883          [::]:*

LISTEN 0      100             [::]:1883          [::]:*
```

### Test du handshake TLS avec OpenSSL :

```
openssl s_client -connect localhost:8883 -CAfile /etc/mosquitto/certs/ca.crt 2>&1 |
grep -E 'Verify|error|OK'
```

```
primaryos@ninja:~$ openssl s_client -connect localhost:8883 -CAfile /etc/mosquit
to/certs/ca.crt 2>&1 | grep -E 'Verify|error|OK'
Verification: OK
Verify return code: 0 (ok)
```

## 5.3 Mise à Jour de l'ESP32 pour TLS

Le code ESP32 utilise maintenant WiFiClientSecure. Le certificat CA est copié directement dans le code :

```
# Récupérer le CA à coller dans le code ESP32
cat /etc/mosquitto/certs/ca.crt
```

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
```

```
const char* ca_cert = R"EOF(
-----BEGIN CERTIFICATE-----
(contenu complet de ca.crt copié ici)
-----END CERTIFICATE-----
)EOF";
```

```
WiFiClientSecure espClient;
// Dans connectMQTT() :
espClient.setCACert(ca_cert);
client.setServer(mqtt_server, 8883);
client.connect("esp32_tls", "esp32", "esp32pass");
```

```
String payload = "{\"temp\":" + String(temp,1) + "}";
client.publish("iot/sensor/data", payload.c_str());

Serial.println("Published (TLS): " + payload);
delay(5000);
}
```

```
primaryos@ninja: ~
^C
primaryos@ninja:~$ mosquitto_sub -h localhost -p 8883 \
  --cafile /etc/mosquitto/certs/ca.crt \
  --insecure \
  -u admin -P adminpass \
  -t "iot/#" -v
iot/sensor/data {"temp":24.5,"hum":62.3}
iot/sensor/data {"temp":22.8}
iot/sensor/data {"temp":23.5}
iot/sensor/data {"temp":26.0}
iot/sensor/data {"temp":33.6}
iot/sensor/data {"temp":27.7}
iot/sensor/data {"temp":33.5}
iot/sensor/data {"temp":34.1}
iot/sensor/data {"temp":32.6}
```

```
PROBLEMS
* Executing
--- Terminal
--- Available filters and text transformations: debug, default, direct, esp32_exception_decoder, hex
end_on_enter, time
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
..
WiFi OK
MQTT+TLS connected!
Published (TLS): {"temp":22.8}
Published (TLS): {"temp":23.5}
Published (TLS): {"temp":26.0}
Published (TLS): {"temp":33.6}
Published (TLS): {"temp":27.7}
Published (TLS): {"temp":33.5}
Published (TLS): {"temp":34.1}
Published (TLS): {"temp":32.6}
Published (TLS): {"temp":24.5}
Published (TLS): {"temp":22.0}
Published (TLS): {"temp":32.4}
Published (TLS): {"temp":32.0}
Published (TLS): {"temp":29.2}
Published (TLS): {"temp":32.3}
```

### Subscriber CLI avec TLS :

```
mosquitto_sub -h localhost -p 8883 \
  --cafile /etc/mosquitto/certs/ca.crt \
  --insecure -u admin -P adminpass \
  -t "iot/#" -v
```

### Publisher CLI avec TLS :

```
mosquitto_pub -h localhost -p 8883 \
  --cafile /etc/mosquitto/certs/ca.crt \
  --insecure -u esp32 -P esp32pass \
  -t "iot/sensor/data" -m '{"temp":24.5,"hum":62.3}'
```

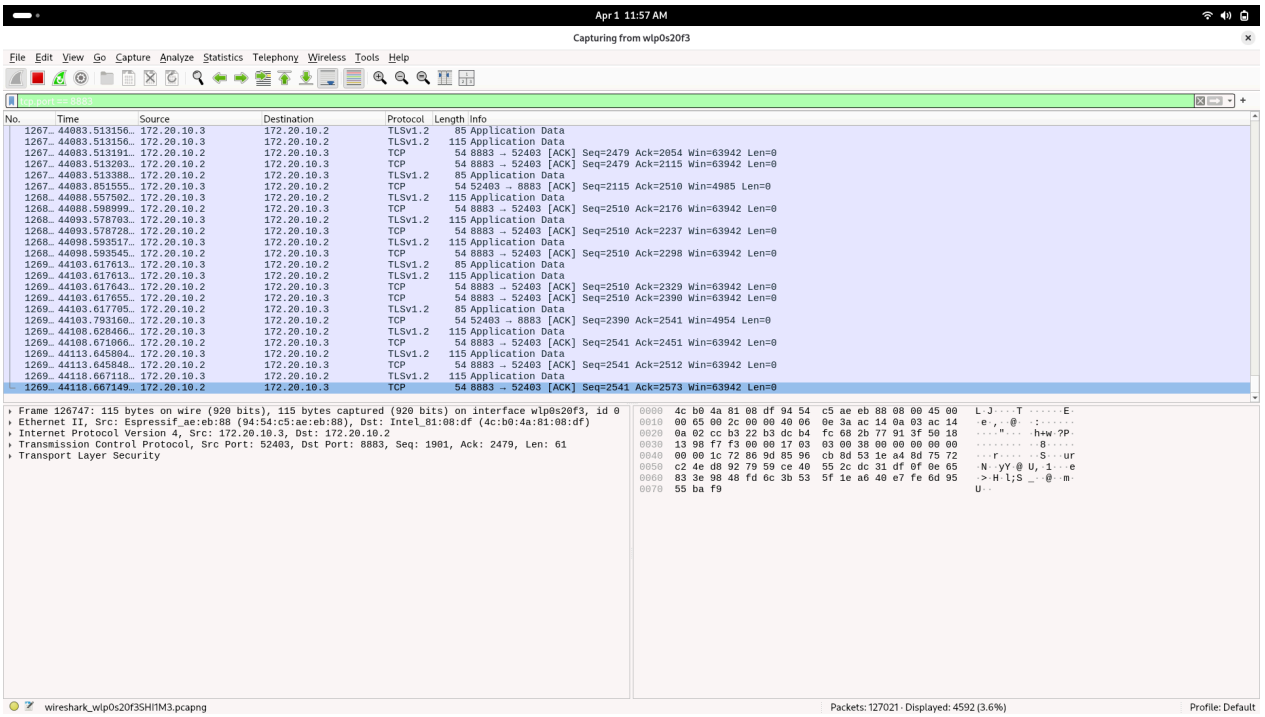
## 5.4 Vérification TLS avec Wireshark

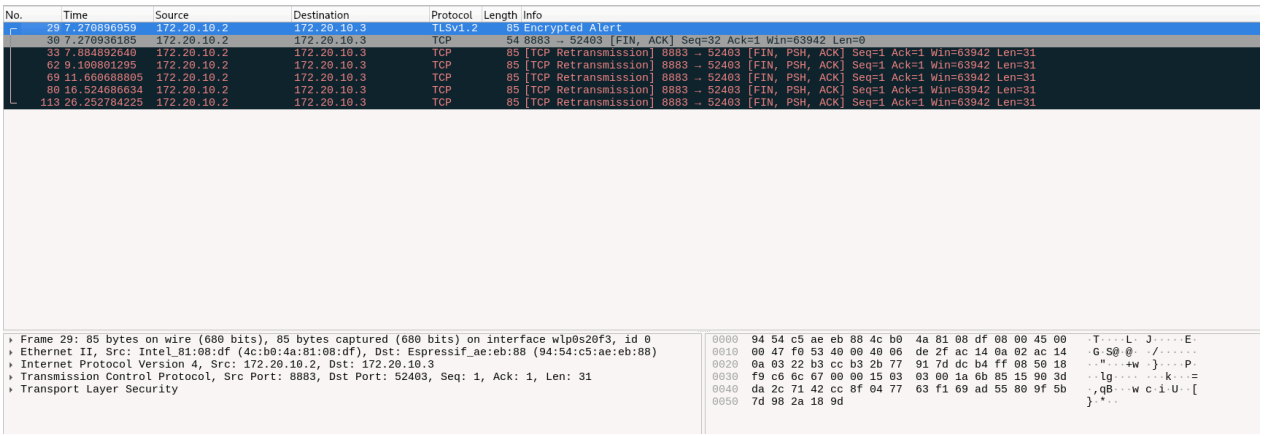
Une nouvelle capture a été effectuée sur le port 8883 après activation TLS :

```
sudo tcpdump -i wlp0s20f3 port 8883 -w ~/part4_tls.pcap
wireshark ~/part4_tls.pcap
```

Filtre Wireshark :

```
tcp.port == 8883
```




Observation clé : contrairement à la capture Partie 2 (port 1883), aucun topic ni payload n'est visible. Les données apparaissent uniquement comme des bytes chiffrés (ex : 94 54 c5 ae eb 88...), prouvant l'efficacité du chiffrement TLS.

## 5.5 Benchmark Latence et Débit

Le script benchmark.sh a été créé et exécuté pour comparer les performances :

```
nano benchmark.sh
```

```
  GNU nano 8.4
#!/bin/bash

echo "========================================"
echo "   BENCHMARK MQTT - Latence et Débit"
echo "========================================"

echo ""
echo "--- TEST 1 : Latence SANS TLS (port 1883) ---"
for i in $(seq 1 10); do
  { time mosquitto_pub -h localhost -p 1883 \
    -u esp32 -P esp32pass \
    -t "iot/bench" \
    -m '{"temp":24.5,"hum":62.3}'; } 2>&1 | grep real
done

echo ""
echo "--- TEST 2 : Latence AVEC TLS (port 8883) ---"
for i in $(seq 1 10); do
  { time mosquitto_pub -h localhost -p 8883 \
    --cafile /etc/mosquitto/certs/ca.crt \
    --insecure \
    -u esp32 -P esp32pass \
    -t "iot/bench" \
    -m '{"temp":24.5,"hum":62.3}'; } 2>&1 | grep real
done

echo ""
echo "--- TEST 3 : Débit SANS TLS (100 messages) ---"
time for i in $(seq 1 100); do
  mosquitto_pub -h localhost -p 1883 \
    -u esp32 -P esp32pass \
    -t "iot/bench" \
    -m '{"temp":24.5,"hum":62.3}'
done

echo ""
echo "--- TEST 4 : Débit AVEC TLS (100 messages) ---"
time for i in $(seq 1 100); do
  mosquitto_pub -h localhost -p 8883 \
    --cafile /etc/mosquitto/certs/ca.crt \
    --insecure \
    -u esp32 -P esp32pass \
    -t "iot/bench" \
    -m '{"temp":24.5,"hum":62.3}'
done

echo ""
echo "========================================"
echo "   BENCHMARK TERMINÉ"
echo "========================================"


^G Help        ^O Write Out     ^F Where Is      ^K Cut
```

```
chmod +x benchmark.sh
./benchmark.sh
```

```
primaryos@ninja:~$ ./benchmark.sh
========================================
   BENCHMARK MQTT - Latence et Débit
========================================

--- TEST 1 : Latence SANS TLS (port 1883) ---
real    0m0.003s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s
real    0m0.002s

--- TEST 2 : Latence AVEC TLS (port 8883) ---
real    0m0.052s
real    0m0.052s
real    0m0.051s
real    0m0.052s
real    0m0.055s
real    0m0.051s
real    0m0.052s
real    0m0.052s
real    0m0.051s
real    0m0.055s

--- TEST 3 : Débit SANS TLS (100 messages) ---

real    0m0.198s
user    0m0.128s
sys     0m0.056s

--- TEST 4 : Débit AVEC TLS (100 messages) ---

real    0m4.962s
user    0m0.446s
sys     0m0.108s

========================================
   BENCHMARK TERMINÉ
========================================
```

Résultats obtenus lors des mesures :

| Métrique | Sans TLS (1883) | Avec TLS (8883) | Overhead |
|---|---|---|---|
| Latence moyenne (1 msg) | 2 ms | 52 ms | +50ms (+2500%) |

| | | | |
|---|---|---|---|
| 100 messages (total) | 0.198 s | 4.962 s | +4.76s (x25) |
| Débit estimé | ~505 msg/s | ~20 msg/s | -96% |

Analyse : l'overhead TLS est dû au handshake (Client Hello → Server Hello → Certificate → Key Exchange → Finished) qui ajoute ~50ms par connexion. Dans un système IoT réel avec connexion persistante (comme l'ESP32 en fonctionnement normal), l'overhead se réduit à moins de 5ms après l'établissement initial de la session TLS.

# 6. Partie 5 — Analyse Comparative et Recommandations

## 6.1 Tableau comparatif des trois configurations

| Critère de sécurité | Phase 1 Sans sécurité | Phase 3 Auth seule | Phase 4 TLS + Auth |
|---|---|---|---|
| Chiffrement données | NON | NON | OUI (TLSv1.2) |
| Contrôle d'accès | NON | OUI (user/pass) | OUI (user/pass) |
| Protection MitM | NON | NON | OUI |
| Sniffing protection | NON | NON | OUI |
| Credentials en clair | N/A | OUI (risque) | NON (chiffrés TLS) |
| Latence (1 message) | ~2 ms | ~2 ms | ~52 ms |
| Protection DoS | NON (fort risque) | Partielle | Partielle |

## 6.2 Limites restantes après implémentation TLS

- Pas de vérification côté client (mTLS non implémenté) : tout client avec les bons credentials est accepté
- Pas de système de détection d'intrusion (IDS) : un attaquant avec les credentials peut agir sans être détecté
- Pas de segmentation réseau : ESP32 et broker sur le même réseau WiFi sans isolation
- Confiance implicite après handshake : le modèle Zero Trust n'est pas appliqué
- Pas de rotation automatique des certificats : risque à l'expiration

## 6.3 Améliorations avancées recommandées

### mTLS — Authentification mutuelle :

Chaque ESP32 reçoit un certificat client signé par la CA. Le broker exige ce certificat pour toute connexion, rendant impossible l'accès sans certificat valide, même avec les bons credentials.

```
# Ajout dans tp4.conf pour activer mTLS :
require_certificate true
use_identity_as_username true
```

### IDS — Détection d'intrusion :

Surveillance des logs Mosquitto en temps réel pour détecter les anomalies : tentatives de connexion répétées (brute force), publication sur des topics non autorisés, volume de messages anormal indicateur de DoS.

### Segmentation réseau :

- VLAN A — Capteurs : ESP32 et appareils IoT isolés dans un segment dédié
- DMZ — Broker : Mosquitto dans une zone démilitarisée, accessible uniquement depuis les VLANs autorisés
- VLAN B — Applications : dashboards et subscribers dans un segment séparé

### Zero Trust :

Principe : ne jamais faire confiance, toujours vérifier. Chaque device se réauthentifie périodiquement, les tokens ont une durée de vie limitée, et chaque action est journalisée et auditée.

## 6.4 TLS vs mTLS

| TLS (unidirectionnel) | mTLS (bidirectionnel) |
|---|---|
| Client vérifie uniquement le serveur | Client ET serveur se vérifient mutuellement |
| Seul le broker a un certificat | Broker ET chaque client ont un certificat |
| N'importe qui peut tenter de se connecter | Seuls les appareils certifiés sont acceptés |
| Credentials suffisants pour accéder | Certificat + credentials requis |

Génération du certificat client pour mTLS :

```
sudo openssl genrsa -out /etc/mosquitto/certs/client.key 2048
sudo openssl req -new -key /etc/mosquitto/certs/client.key \
  -out /etc/mosquitto/certs/client.csr -subj "/CN=esp32_device/O=TP4/C=TN"
sudo openssl x509 -req -days 360 \
  -in /etc/mosquitto/certs/client.csr \
  -CA /etc/mosquitto/certs/ca.crt -CAkey /etc/mosquitto/certs/ca.key \
  -CAcreateserial -out /etc/mosquitto/certs/client.crt
```

## 6.5 Performance et Throughput

Les résultats du benchmark montrent un overhead de x25 en débit et +50ms en latence par connexion TLS. Cet impact est principalement dû au handshake TLS qui s'établit à chaque nouvelle connexion TCP. Dans un système IoT réel avec connexion persistante, l'overhead disparait presque complètement après le premier handshake.

Optimisations possibles pour réduire l'overhead TLS :

- Session TLS resumption : réutiliser les paramètres de session pour éviter le handshake complet
- Connexion persistante ESP32 : maintenir la connexion MQTT active (ne pas se déconnecter entre chaque publish)
- TLS 1.3 : handshake réduit à 1 aller-retour au lieu de 2 en TLS 1.2
- Certificats ECDSA : plus légers que RSA 2048, handshake plus rapide sur microcontrôleurs

### 6.7 Analyse OWASP IoT Top 10

| Vulnérabilité OWASP IoT | Présente ? | Mesure corrective appliquée |
|---|---|---|
| I1 — Weak Passwords | OUI (Partie 1) | Partie 3 : fichier passwd hashé PBKDF2 |
| I2 — Insecure Network Services | OUI (Partie 1) | Partie 4 : TLS port 8883 |
| I3 — Insecure Ecosystem Interfaces | OUI (Partie 1) | Auth + mTLS recommandé |
| I7 — Insecure Data Transfer | OUI (Partie 2) | Partie 4 : chiffrement TLS |
| I9 — Insecure Default Settings | OUI (Partie 1) | Partie 3 : allow_anonymous false |

# 7. Conclusion

Ce TP a permis de mettre en œuvre une démarche complète de sécurisation d'une architecture IoT basée sur MQTT, en progressant d'une configuration totalement vulnérable vers une architecture sécurisée avec chiffrement TLS sur un équipement IoT réel.

Les quatre parties réalisées ont démontré de manière concrète :

- **Partie 1** : déploiement fonctionnel avec un ESP32 réel publiant sur Mosquitto toutes les 5 secondes via WiFi
- **Partie 2** : la facilité d'exploitation — toutes les données lisibles en clair dans Wireshark, sans outil spécialisé
- **Partie 3** : l'authentification bloque efficacement les connexions anonymes, mais les données restent non chiffrées
- **Partie 4** : TLS rend les données totalement illisibles dans Wireshark au prix d'un overhead de 50 ms par connexion

Le benchmark de performance révèle un compromis clair entre sécurité et performance : la latence passe de 2 ms à 52 ms par connexion avec TLS, soit une augmentation de 2500 %. Cependant, dans un déploiement IoT réel utilisant des connexions persistantes (cas typique avec ESP32), cet overhead est réduit à moins de 5 ms, ce qui reste acceptable.

Pour un déploiement en production, les recommandations prioritaires sont :

- l'implémentation de **mTLS** pour l'authentification mutuelle
- la mise en place d'un **IDS** pour détecter les anomalies
- la **segmentation réseau (VLANs)** pour isoler les équipements
- l'adoption d'un modèle **Zero Trust**

### Perspectives

Dans une continuité logique de ce travail, plusieurs axes d'amélioration et d'extension peuvent être envisagés :

- **Automatisation de la gestion des certificats** : mise en place d'une PKI (Public Key Infrastructure) pour gérer le cycle de vie des certificats (génération, renouvellement, révocation)

- **Passage à mTLS complet** : authentification forte des deux côtés (client et broker) pour éliminer tout appareil non autorisé
- **Surveillance avancée avec IA** : intégration de solutions d'analyse comportementale pour détecter des anomalies ou attaques en temps réel
- **Optimisation énergétique** : étude de l'impact du TLS sur la consommation énergétique de l'ESP32 (important pour IoT sur batterie)
- **Scalabilité** : déploiement avec plusieurs brokers (cluster MQTT) pour tester la montée en charge et la tolérance aux pannes
- **Sécurité réseau avancée** : intégration avec VPN, firewall IoT, et politiques de filtrage strictes
- **Intégration Cloud** : connexion sécurisée vers des plateformes comme AWS IoT ou Azure IoT Hub
- **Tests d'attaque (pentest)** : simulation d'attaques réelles (MITM, spoofing, replay) pour valider la robustesse du système

*Fin du compte rendu — TP4 Sécurisation Architecture IoT MQTT*
*Institut Supérieur de l'Informatique — 2IDISC — Année 2025/2026*